Unsupervised Learning and Model Evaluation

Week 3 Task Report - Clustering and Model Evaluation Report

Task	Perform clustering on a dataset and prepare a model evaluation report
Prepared by	Shubh
Date	28 October 2026
Tools	Python 3, Scikit-learn, SciPy, Pandas, NumPy, Matplotlib
Data	Cleaned loan applicant dataset from Week 1 (515 rows, 15 columns)
Part A	Unsupervised learning: PCA, K-Means and Hierarchical clustering, cluster interpretation
Part B	Model evaluation: cross-validation, confusion matrix, precision, recall, F1, ROC-AUC, hyperparameter tuning, decision threshold
Part C	Model analysis: feature importance across models, why the models perform differently (ablation, learning curves, significance test, noise ceiling), error analysis
Code	src/clustering_and_evaluation.py (full code and results in the GitHub repository)

Contents

- 1. Objective and overview
- 2. Key concepts
- 3. Step-by-step documentation (Steps 1 to 12)
- 4. Discussion: why the models perform differently and how feature importance varies
- 5. Summary of findings
- 6. Limitations and next steps
- Appendix: repository and how to run

1. Objective and overview

This report covers the Week 3 objectives:

- Understand clustering algorithms (K-Means and Hierarchical Clustering) and dimensionality reduction (PCA).
- Learn cross-validation, the confusion matrix, precision, recall, F1-score, ROC-AUC and hyperparameter tuning.
- **Deliverable:** perform clustering on a dataset and prepare a model evaluation report.

The work is split into three parts using the cleaned loan applicant dataset from Weeks 1 and 2. **Part A** (Steps 1-5) clusters the applicants into groups without using the approval label and interprets the groups. **Part B** (Steps 6-9) evaluates classification models that predict loan approval, with a focus on *how* a model should be judged: a baseline for comparison, cross-validation for stable scores, tuning, and the choice of decision threshold. **Part C** (Steps 10-12 and Section 4) explains *why* the models perform as they do: which features each model relies on, what separates the best model from the weakest, whether the differences are real, how close the models are to the best score the data allows, and which applicants are misclassified.

2. Key concepts

Concept	In one sentence
Unsupervised learning	Finds structure in data that has no labels (no correct answer to learn from).
K-Means	Splits data into k groups around k centres that are moved until the groups are stable. Needs k in advance and assumes round, similarly sized groups.
Hierarchical clustering	Builds a tree by merging the closest clusters step by step; the tree can be cut at any height. No need to choose k in advance.
PCA	Rotates the data to new axes (components) ordered by how much variation they capture, so a few components can summarise many features.
Silhouette score	-1 to 1: how well each point fits its own cluster versus the next nearest. Above 0.5 is clear structure; below 0.25 is weak.
Cross-validation	Repeats training and testing on different splits of the data to get a stable score and its spread.
Confusion matrix	Table of true positives, true negatives, false positives and false negatives.
Precision / Recall / F1	Precision: how many predicted positives are right. Recall: how many real positives are found. F1: the balance of both.
ROC-AUC	How well the model ranks positives above negatives over all thresholds: 0.5 is guessing, 1.0 is perfect.
Hyperparameter tuning	Searching for the best model settings with cross-validation (grid or random search) without touching the test set.
Permutation importance	Shuffles one feature and measures how much the model's score drops; works for any model and shows what the model really relies on.
Learning curve	Training and validation score at growing training sizes; shows whether a model memorises (large gap) and whether more data would help (validation still rising).
Feature ablation	Re-running a model with fewer (or engineered) features to see which features help or hurt.
Noise ceiling	The best score any model can reach when the outcome is partly random; a model cannot be blamed for missing what is not predictable.

3. Step-by-step documentation

Each step shows what is done and why, the exact code that was run, its output, and the result. The code comes straight from **src/clustering_and_evaluation.py**.

Part A - Unsupervised learning (clustering)

Step 1: Load the data and choose the clustering features

Clustering is **unsupervised**: there is no target column to predict, the algorithm looks for natural groups on its own. The label *Approved* is therefore **not used**. Only the six columns that describe an applicant are used (age, education level, income, credit score, loan amount, loan term). The 0/1 dummy columns are left out because they would dominate the distance calculation. Clustering is distance-based, so all features are first put on the same scale with **StandardScaler** (mean 0, standard deviation 1).

CODE

```
df = pd.read_csv("data/loan_applicants_clean.csv")
print("Shape:", df.shape)
# Clustering is unsupervised: the label (Approved) is NOT used. Only the six numeric / ordinal
# columns that describe the applicant's profile are used; 0/1 dummy columns would dominate distances.
features = ["Age", "Education", "Annual_Income", "Credit_Score", "Loan_Amount", "Loan_Term_Months"]
X_scaled = StandardScaler().fit_transform(df[features])   # clustering needs all features on one scale
print("Clustering features:", features)
print("Scaled matrix shape:", X_scaled.shape)
print("\nAverage and spread of the scaled features (should be 0 and 1):")
print(pd.DataFrame(X_scaled, columns=features).agg(["mean", "std"]).round(2))
```

OUTPUT

```
Shape: (515, 15)
Clustering features: ['Age', 'Education', 'Annual_Income', 'Credit_Score', 'Loan_Amount', 'Loan_Term_Months']
Scaled matrix shape: (515, 6)

Average and spread of the scaled features (should be 0 and 1):
      Age  Education  Annual_Income  Credit_Score  Loan_Amount  Loan_Term_Months
mean  0.0       -0.0            0.0           0.0          0.0               0.0
std   1.0        1.0            1.0           1.0          1.0               1.0
```

Result: 515 applicants described by 6 scaled features.

Step 2: Dimensionality reduction with PCA

Dimensionality reduction compresses many features into a few new ones, called principal components, that keep as much information (variance) as possible. **PCA** finds the directions in which the data varies most. It is used here to see how many independent patterns exist and to draw the data in 2-D. The loadings show which original features build each component.

CODE

```
pca = PCA(n_components=len(features), random_state=42).fit(X_scaled)
evr = pca.explained_variance_ratio_
print(pd.DataFrame({"Explained": evr.round(3), "Cumulative": evr.cumsum().round(3)},
                   index=[f"PC{i + 1}" for i in range(len(evr))]))
n95 = int(np.argmax(evr.cumsum() >= 0.95)) + 1
print(f"\nComponents needed to keep 95% of the information: {n95} of {len(features)}")
print("\nWhat PC1 and PC2 are made of (loadings):")
print(pd.DataFrame(pca.components_[:2].T, index=features, columns=["PC1", "PC2"]).round(2))
pc2 = PCA(n_components=2, random_state=42).fit_transform(X_scaled)   # 2-D view used for plotting
plot_pca_variance(evr)
```

OUTPUT

```
     Explained  Cumulative
PC1      0.410       0.410
PC2      0.177       0.587
PC3      0.156       0.743
PC4      0.141       0.884
PC5      0.092       0.976
PC6      0.024       1.000

Components needed to keep 95% of the information: 5 of 6

What PC1 and PC2 are made of (loadings):
                   PC1   PC2
Age               0.50  0.39
Education         0.33 -0.38
```

```
Annual_Income     0.58  0.07
Credit_Score      0.27 -0.39
Loan_Amount       0.48  0.09
Loan_Term_Months -0.07  0.73
```

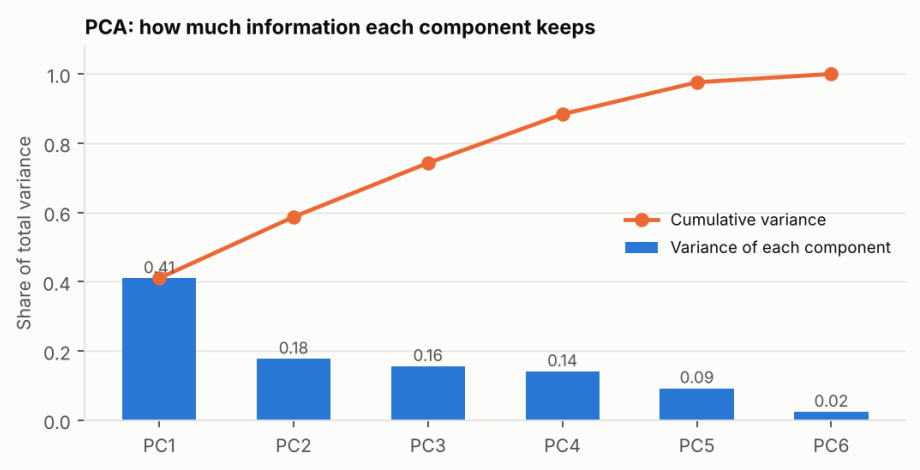


Figure 1 - Variance kept by each principal component (bars) and in total (line).

Result: PC1 keeps 41% and PC2 18% of the variance, so a 2-D plot shows about 59% of the information. 5 of 6 components are needed for 95%, which means the features carry mostly separate information and the data cannot be squeezed much further. PC1 is an overall 'size' axis: income, age and loan amount rise together.

Step 3: K-Means clustering and choosing k

K-Means places k centres, assigns every applicant to the nearest centre, moves each centre to the average of its members, and repeats until nothing changes. k must be chosen. Two guides are used: the **elbow method** (inertia, the total distance to the centres, always falls as k grows; look for a bend) and the **silhouette score** (from -1 to 1: how much closer an applicant is to its own cluster than to the nearest other one; higher is better).

CODE

```
rows = []
for k in range(2, 9):
    km = KMeans(n_clusters=k, n_init=10, random_state=42).fit(X_scaled)
    rows.append({"k": k, "Inertia": km.inertia_,
                 "Silhouette": silhouette_score(X_scaled, km.labels_),
                 "Davies_Bouldin": davies_bouldin_score(X_scaled, km.labels_)})
k_table = pd.DataFrame(rows).set_index("k")
print(k_table.round(3))
best_k = int(k_table["Silhouette"].idxmax())          # highest silhouette = best separated clusters
km = KMeans(n_clusters=best_k, n_init=10, random_state=42).fit(X_scaled)
km_labels = km.labels_
print(f"\nChosen k = {best_k} (highest silhouette score)")
print("Applicants per cluster:")
print(pd.Series(km_labels).value_counts().sort_index())
plot_k_selection(k_table, best_k)
k_table.to_csv(f"{OUT}/kmeans_k_selection.csv")
```

OUTPUT

```
    Inertia  Silhouette  Davies_Bouldin
k
2  2242.951       0.234           1.565
3  1973.595       0.178           1.756
4  1758.723       0.174           1.687
5  1617.364       0.176           1.620
6  1510.777       0.166           1.624
7  1419.423       0.164           1.603
8  1325.630       0.161           1.553

Chosen k = 2 (highest silhouette score)
Applicants per cluster:
0    253
1    262
Name: count, dtype: int64
```

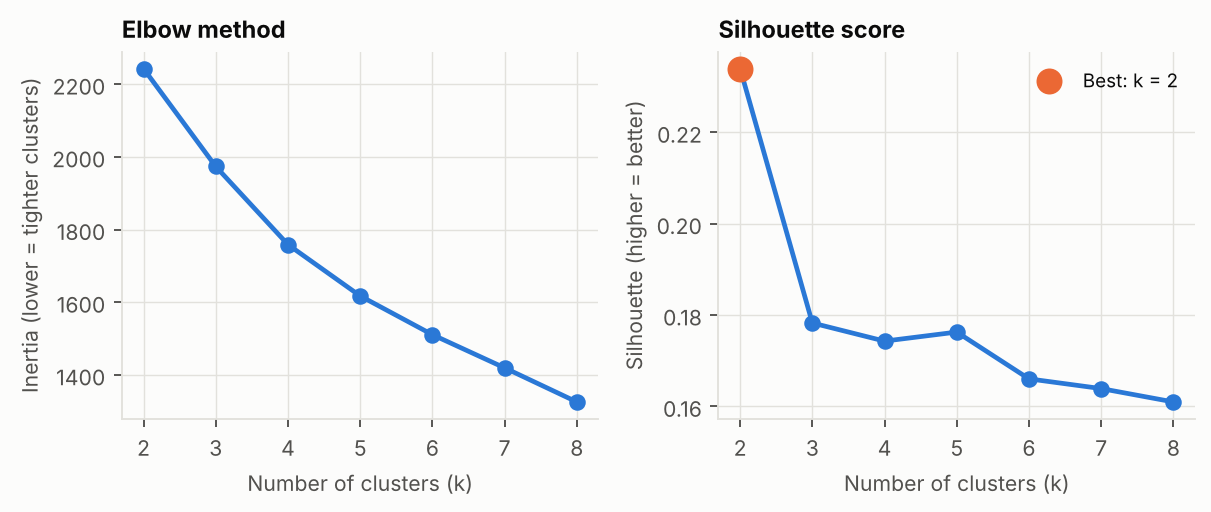


Figure 2 - Elbow curve (left) and silhouette score (right) for k = 2 to 8.

Result: The silhouette score is highest at k = 2 (0.234) and the elbow curve has no clear bend, so k = 2 is chosen. **Important:** a silhouette of about 0.23 is low. Values above 0.5 mean clearly separated groups; here the applicants form one continuous cloud, and the clusters split it into two halves rather than finding natural islands.

Step 4: Hierarchical clustering

Hierarchical clustering starts with every applicant as its own cluster and repeatedly merges the two closest clusters, building a tree called a **dendrogram**. Cutting the tree at a chosen height gives the clusters. The **linkage** rule decides how distance between clusters is measured; **Ward** merges the pair that increases the within-cluster variance least. The cluster numbers of the two methods are matched with the Hungarian algorithm, and the **Adjusted Rand Index (ARI)** measures how much they agree (1 = identical, 0 = random).

CODE

```
Z = linkage(X_scaled, method="ward")                # bottom-up merge tree
rows = []
for method in ["ward", "complete", "average", "single"]:
    lab = AgglomerativeClustering(n_clusters=best_k, linkage=method).fit_predict(X_scaled)
    rows.append({"Linkage": method, "Silhouette": silhouette_score(X_scaled, lab),
                 "ARI_vs_KMeans": adjusted_rand_score(km_labels, lab),
                 "Smallest_cluster": int(np.bincount(lab).min())})
print(pd.DataFrame(rows).set_index("Linkage").round(3))
hc_labels = AgglomerativeClustering(n_clusters=best_k, linkage="ward").fit_predict(X_scaled)
# Cluster numbers are arbitrary: renumber the hierarchical clusters to match K-Means
cont = pd.crosstab(km_labels, hc_labels).to_numpy()
r, c = linear_sum_assignment(-cont)
mapping = {int(ci): int(ri) for ri, ci in zip(r, c)}
hc_labels = np.array([mapping[int(l)] for l in hc_labels])
print("\nK-Means (rows) vs Hierarchical (columns):")
print(pd.crosstab(pd.Series(km_labels, name="KMeans"), pd.Series(hc_labels, name="Hierarchical")))
ari = adjusted_rand_score(km_labels, hc_labels)
print(f"\nAdjusted Rand Index between the two methods: {ari:.3f}  (1.0 = identical clusters)")
plot_dendrogram(Z, best_k)
plot_clusters(pc2, km_labels, hc_labels, evr, best_k)
```

OUTPUT

```
           Silhouette  ARI_vs_KMeans  Smallest_cluster
Linkage
ward            0.170          0.336               198
complete        0.224          0.412               167
average         0.244          0.001                 5
single          0.265          0.000                 1

K-Means (rows) vs Hierarchical (columns):
Hierarchical    0    1
KMeans
0             231   22
1              86  176

Adjusted Rand Index between the two methods: 0.336  (1.0 = identical clusters)
```

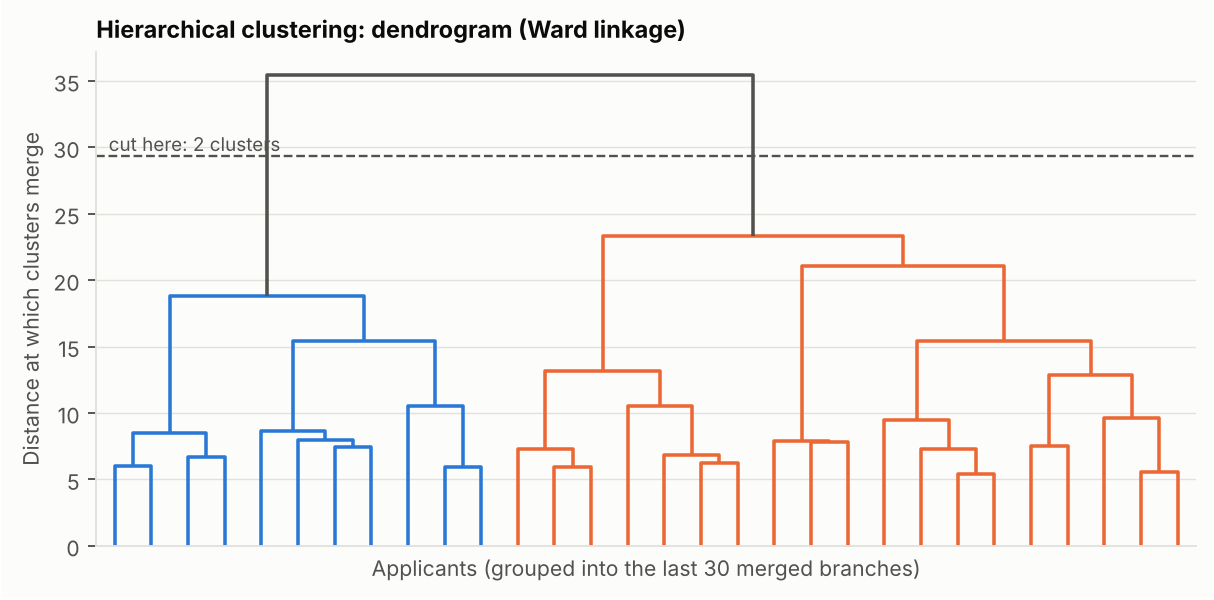


Figure 3 - Dendrogram. The dashed line shows where the tree is cut to get the clusters.

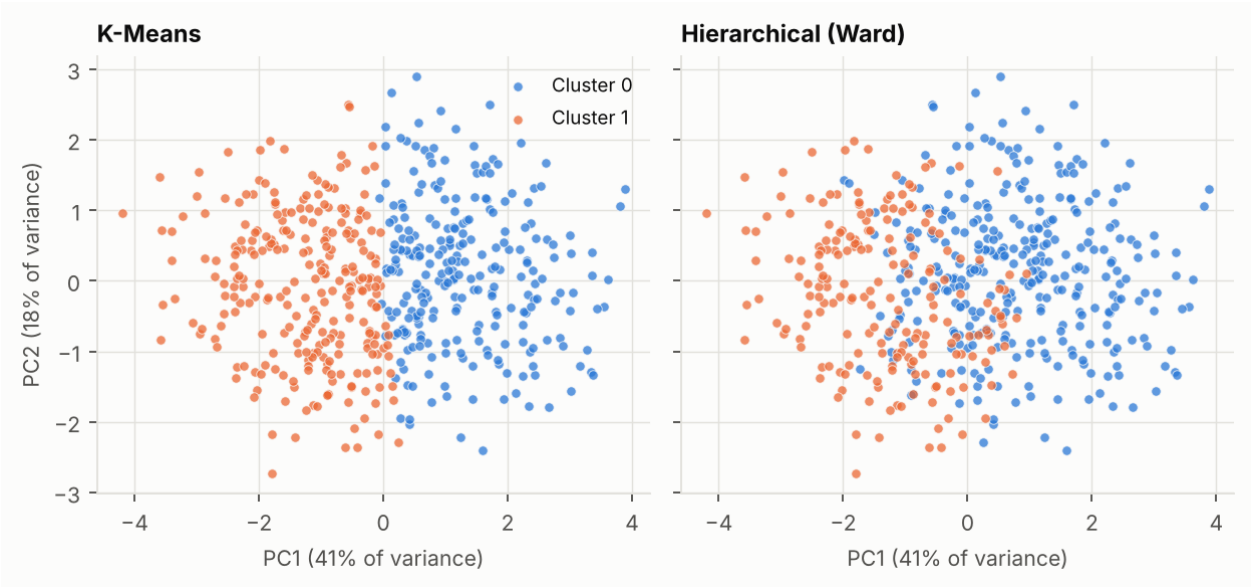


Figure 4 - The two clusterings drawn on the first two principal components. The clusters touch and overlap: there is no empty gap between them.

Result: Ward linkage gives balanced clusters (198+ applicants each). The single and average linkages reach a higher silhouette score only because they create tiny or one-person clusters (smallest cluster: 1 and 5 applicants), which is useless. The ARI between K-Means and Ward is 0.336, a **moderate** agreement: the two methods draw the boundary in somewhat different places, another sign that the groups are not sharply separated.

Step 5: Describe and interpret the clusters

A cluster is only useful if it can be described. The average of every feature is computed per cluster and converted back to real units with the Week 1 means and standard deviations. The approval rate per cluster is an **independent check**, because the label was never given to the algorithm.

CODE

```
# Convert the z-scores back to real units with the means / standard deviations from Week 1 (Step 11)
SCALE = {"Age": (42.9, 12.2), "Annual_Income": (778441.6, 227053.6), "Credit_Score": (609.5, 49.0),
         "Loan_Amount": (689110.7, 365853.3), "Loan_Term_Months": (36.3, 17.0)}
profile_z = df[features].groupby(km_labels).mean()
profile = profile_z.copy()
for col, (mu, sd) in SCALE.items():
    profile[col] = profile_z[col] * sd + mu
profile.insert(0, "Size", pd.Series(km_labels).value_counts().sort_index())
profile["Approval_%"] = (df["Approved"].groupby(km_labels).mean() * 100).round(1)
print("Cluster profiles in real units (Education: 0 = High School ... 3 = PhD):")
print(profile.round({"Age": 1, "Education": 2, "Annual_Income": 0, "Credit_Score": 0,
                     "Loan_Amount": 0, "Loan_Term_Months": 1}).T)
print("\nThe approval label was NOT used for clustering, so a difference in approval rate")
print("between clusters is an independent check that the clusters are meaningful.")
plot_profiles(profile_z)
profile.round(2).to_csv(f"{OUT}/cluster_profiles.csv")
labelled = df.copy()
labelled["KMeans_Cluster"], labelled["Hierarchical_Cluster"] = km_labels, hc_labels
labelled.to_csv(f"{OUT}/clustered_data.csv", index=False)
```

OUTPUT

```
Cluster profiles in real units (Education: 0 = High School ... 3 = PhD):
                         0          1
Size                 253.0     262.00
Age                   51.7      34.40
Education              1.5       0.97
Annual_Income     954572.0  608361.00
Credit_Score         625.0     595.00
Loan_Amount       911530.0  474332.00
Loan_Term_Months      35.3      37.20
Approval_%            68.4      54.60

The approval label was NOT used for clustering, so a difference in approval rate
between clusters is an independent check that the clusters are meaningful.
```

Cluster profiles (K-Means)

Cluster	Size	Avg age	Education (0-3)	Avg income (Rs.)	Credit score	Avg loan (Rs.)	Approval rate
Cluster 0	253	51.7	1.50	954,572	625	911,530	68.4%
Cluster 1	262	34.4	0.97	608,361	595	474,332	54.6%

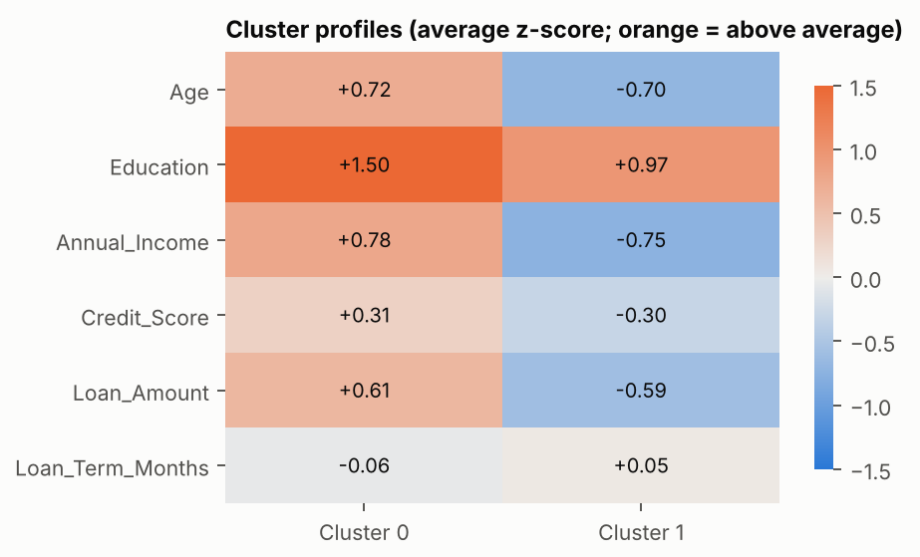


Figure 5 - How each cluster differs from the average applicant, per feature.

Result: Cluster 0 (253 applicants) are older, more educated, higher-income applicants with larger loans (average age 52, income Rs. 954,572). **Cluster 1** (262) are younger, lower-income applicants with smaller loans (age 34, income Rs. 608,361). Approval is 68% in cluster 0 versus 55% in cluster 1, so the grouping carries real information about approval even though approval was not used.

Part B - Model evaluation

Step 6: Model evaluation - cross-validation

A single train/test split can be lucky or unlucky. **Cross-validation** repeats the test many times: the training data is cut into 5 folds, each fold is used once for validation, and the whole process is repeated 3 times with different cuts (15 rounds). The mean shows the typical score and the **standard deviation** shows how stable it is. A **baseline** that always predicts 'approved' is included, because a model is only useful if it beats the trivial answer. The gap between the training score and the validation score reveals **overfitting**.

CODE

```
X, y = df.drop(columns="Approved"), df["Approved"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42, stratify=y)
print("Train / test rows:", len(X_train), "/", len(X_test))
models = {
    "Baseline (always approve)": DummyClassifier(strategy="most_frequent"),
    "Logistic Regression": LogisticRegression(max_iter=1000),
    "KNN": KNeighborsClassifier(n_neighbors=7),
    "Random Forest": RandomForestClassifier(n_estimators=200, max_depth=8, random_state=42),
}
scoring = ["accuracy", "precision", "recall", "f1", "roc_auc"]
rcv = RepeatedStratifiedKFold(n_splits=5, n_repeats=3, random_state=42)   # 15 train/validate rounds
rows = []
for name, model in models.items():
    cv = cross_validate(model, X_train, y_train, cv=rcv, scoring=scoring, return_train_score=True)
    row = {"Model": name, "train_auc": cv["train_roc_auc"].mean()}
    for s in scoring:
        row[f"{s}_mean"], row[f"{s}_std"] = cv[f"test_{s}"].mean(), cv[f"test_{s}"].std()
    rows.append(row)
cv_summary = pd.DataFrame(rows).set_index("Model")
show = pd.DataFrame({s: cv_summary[f"{s}_mean"].map("{:.3f}".format) + " +/- " +
                        cv_summary[f"{s}_std"].map("{:.3f}".format) for s in scoring})
print("\nCross-validation results (mean +/- standard deviation over 15 folds):")
print(show)
print("\nOverfitting check - AUC on the training folds vs on the validation folds:")
print(pd.DataFrame({"Train_AUC": cv_summary["train_auc"], "Validation_AUC": cv_summary["roc_auc_mean"]}).round(3))
plot_cv(cv_summary)
cv_summary.round(4).to_csv(f"{OUT}/cross_validation_summary.csv")
```

OUTPUT

```
Train / test rows: 412 / 103

Cross-validation results (mean +/- standard deviation over 15 folds):
                               accuracy        precision           recall               f1  \
Model
Baseline (always approve)  0.614 +/- 0.004  0.614 +/- 0.004  1.000 +/- 0.000  0.761 +/- 0.003
Logistic Regression        0.689 +/- 0.043  0.719 +/- 0.027  0.809 +/- 0.055  0.761 +/- 0.036
KNN                        0.653 +/- 0.038  0.684 +/- 0.029  0.809 +/- 0.047  0.741 +/- 0.030
Random Forest              0.670 +/- 0.045  0.692 +/- 0.029  0.836 +/- 0.063  0.756 +/- 0.037

                                roc_auc
Model
Baseline (always approve)  0.500 +/- 0.000
Logistic Regression        0.732 +/- 0.065
KNN                        0.654 +/- 0.063
Random Forest              0.711 +/- 0.055

Overfitting check - AUC on the training folds vs on the validation folds:
                           Train_AUC  Validation_AUC
Model
Baseline (always approve)      0.500           0.500
Logistic Regression            0.777           0.732
KNN                            0.789           0.654
Random Forest                  1.000           0.711
```

Cross-validation results (mean +/- standard deviation over 15 folds)

Model	Accuracy	Precision	Recall	F1	ROC-AUC	Train AUC
Baseline (always approve)	0.614 +/- 0.004	0.614 +/- 0.004	1.000 +/- 0.000	0.761 +/- 0.003	0.500 +/- 0.000	0.500
Logistic Regression	0.689 +/- 0.043	0.719 +/- 0.027	0.809 +/- 0.055	0.761 +/- 0.036	0.732 +/- 0.065	0.777
KNN	0.653 +/- 0.038	0.684 +/- 0.029	0.809 +/- 0.047	0.741 +/- 0.030	0.654 +/- 0.063	0.789
Random Forest	0.670 +/- 0.045	0.692 +/- 0.029	0.836 +/- 0.063	0.756 +/- 0.037	0.711 +/- 0.055	1.000

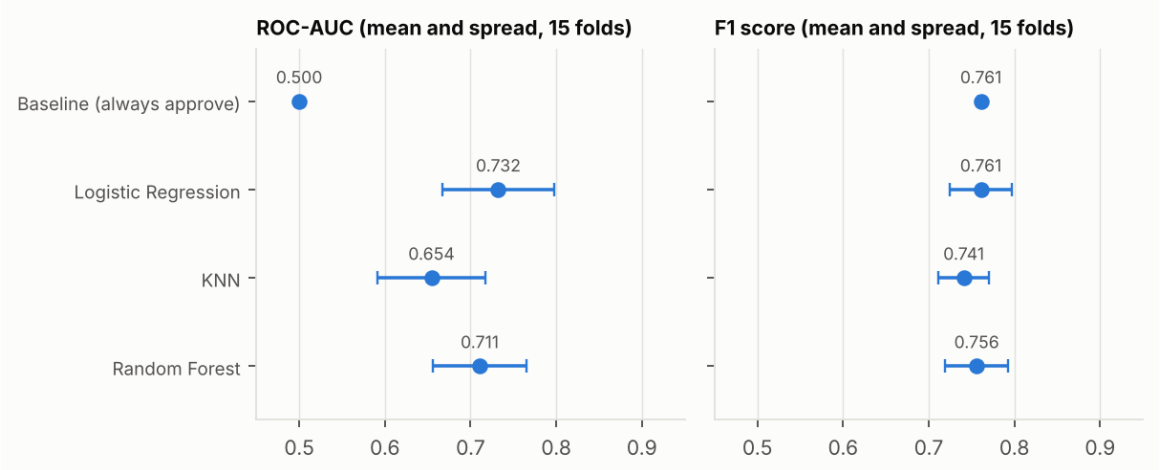


Figure 6 - Cross-validated scores. The lines show the standard deviation over 15 folds.

Result: The always-approve baseline already reaches accuracy 0.614 and an F1 of 0.761, because 61% of applicants are approved. Logistic Regression matches the baseline F1 (0.761) and clearly beats it on accuracy and ROC-AUC (0.732 vs 0.500). **ROC-AUC is the metric that shows real skill here**, because accuracy and F1 flatter a model on an unbalanced label. The Random Forest memorises the training folds (training AUC 1.000 vs validation 0.711): a clear case of overfitting. The spread of about 0.06 AUC between folds shows that differences smaller than that are noise.

Step 7: Hyperparameter tuning

Hyperparameters are settings chosen before training (number of neighbours, tree depth, regularisation strength). **GridSearchCV** tries every combination with 5-fold cross-validation and keeps the best, scored by ROC-AUC on the **training data only**. **RandomizedSearchCV** tries a fixed number of random combinations from a much wider range, which is cheaper when there are many settings. A **validation curve** varies one setting and plots the training score against the validation score.

CODE

```
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
grids = {
    "Logistic Regression": (LogisticRegression(max_iter=1000), {"C": [0.01, 0.1, 1, 10]}),
    "KNN": (KNeighborsClassifier(), {"n_neighbors": [5, 11, 21, 31], "weights": ["uniform", "distance"]}),
    "Random Forest": (RandomForestClassifier(random_state=42),
                      {"n_estimators": [100, 200], "max_depth": [3, 4, 6, 8], "min_samples_leaf": [1, 3, 5]}),
}
tuned, rows, rf_search = {}, [], None
for name, (model, grid) in grids.items():
    gs = GridSearchCV(model, grid, cv=skf, scoring="roc_auc").fit(X_train, y_train)
    tuned[name] = gs.best_estimator_
    n_comb = len(gs.cv_results_["params"])
    rows.append({"Model": name, "Search": "Grid", "Combinations": n_comb, "Model_fits": n_comb * 5,
                 "Best_parameters": str(gs.best_params_), "CV_AUC": gs.best_score_})
    if name == "Random Forest":
        rf_search = gs
tuned_table = pd.DataFrame(rows).set_index("Model")
print("GridSearchCV tries every combination (5-fold CV, scored by ROC-AUC):")
print(tuned_table[["Combinations", "Model_fits", "CV_AUC"]].round(3))
print()
for name, row in tuned_table.iterrows():
    print(f"{name:20s} best: {row['Best_parameters']}")

res = pd.DataFrame(rf_search.cv_results_)
top = pd.concat([pd.DataFrame(list(res["params"])), res[["mean_test_score", "std_test_score"]]], axis=1)
```

```
print("\nTop 5 Random Forest combinations from the grid:")
print(top.sort_values("mean_test_score", ascending=False).head(5).round(3).to_string(index=False))

# RandomizedSearchCV: samples a fixed number of random combinations from a much wider space
rs = RandomizedSearchCV(
    RandomForestClassifier(random_state=42),
    {"n_estimators": randint(50, 400), "max_depth": randint(2, 12), "min_samples_leaf": randint(1, 15),
     "max_features": ["sqrt", "log2", None]},
    n_iter=20, cv=skf, scoring="roc_auc", random_state=42).fit(X_train, y_train)
print("\nGridSearchCV vs RandomizedSearchCV for the Random Forest:")
print(pd.DataFrame({"Fits": [24 * 5, 20 * 5],
                    "Best_CV_AUC": [rf_search.best_score_, rs.best_score_]},
                   index=["GridSearchCV (24 combinations)", "RandomizedSearchCV (20 random draws)"]).round(3))
print("Randomized best parameters:", rs.best_params_)

# Validation curve: one hyperparameter at a time, training score vs validation score
depths = [1, 2, 3, 4, 6, 8, 10, 12]
tr_s, va_s = validation_curve(RandomForestClassifier(n_estimators=100, random_state=42), X_train, y_train,
                              param_name="max_depth", param_range=depths, cv=skf, scoring="roc_auc")
print("\nValidation curve (Random Forest max_depth):")
print(pd.DataFrame({"Train_AUC": tr_s.mean(axis=1), "Validation_AUC": va_s.mean(axis=1)},
                   index=pd.Index(depths, name="max_depth")).round(3))
plot_validation(depths, tr_s, va_s)
tuned_table.to_csv(f"{OUT}/tuning_results.csv")
```

OUTPUT

```
GridSearchCV tries every combination (5-fold CV, scored by ROC-AUC):
                     Combinations  Model_fits  CV_AUC
Model
Logistic Regression             4          20   0.738
KNN                             8          40   0.717
Random Forest                  24         120   0.724

Logistic Regression  best: {'C': 0.1}
KNN                  best: {'n_neighbors': 31, 'weights': 'distance'}
Random Forest        best: {'max_depth': 4, 'min_samples_leaf': 5, 'n_estimators': 200}

Top 5 Random Forest combinations from the grid:
 max_depth  min_samples_leaf  n_estimators  mean_test_score  std_test_score
         4                 5           200            0.724           0.063
         6                 5           100            0.722           0.071
         4                 3           200            0.721           0.065
         4                 5           100            0.721           0.068
         6                 5           200            0.721           0.062

GridSearchCV vs RandomizedSearchCV for the Random Forest:
                                    Fits  Best_CV_AUC
GridSearchCV (24 combinations)         120        0.724
RandomizedSearchCV (20 random draws)   100        0.733
Randomized best parameters: {'max_depth': 2, 'max_features': None, 'min_samples_leaf': 11, 'n_estimators': 219}

Validation curve (Random Forest max_depth):
           Train_AUC  Validation_AUC
max_depth
1              0.775           0.688
2              0.828           0.704
3              0.870           0.708
4              0.912           0.718
6              0.980           0.717
8              0.999           0.720
10             1.000           0.712
12             1.000           0.704
```

Tuning results

Model	Search	Combinations	Model fits	Best parameters	CV AUC
Logistic Regression	Grid	4	20	{'C': 0.1}	0.738
KNN	Grid	8	40	{'n_neighbors': 31, 'weights': 'distance'}	0.717
Random Forest	Grid	24	120	{'max_depth': 4, 'min_samples_leaf': 5, 'n_estimators': 200}	0.724

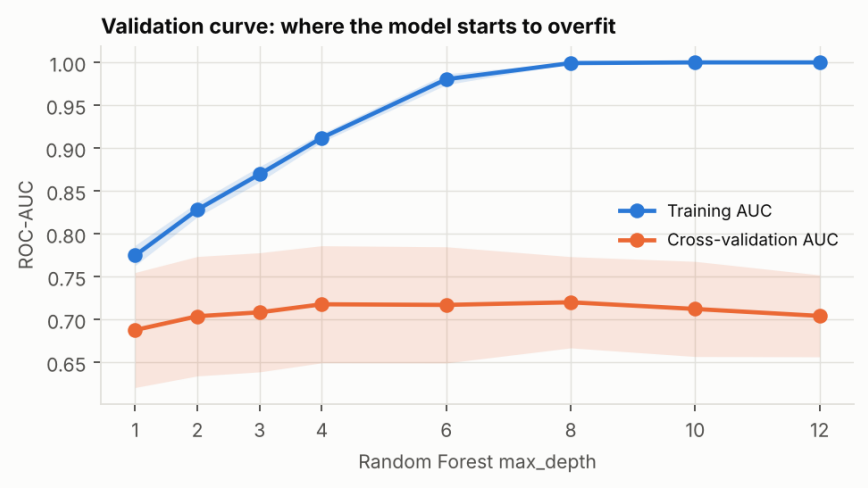


Figure 7 - Training score keeps rising with depth; validation score stays flat.

Result: Best settings: Logistic Regression C = 0.1, KNN k = 31 with distance weighting, Random Forest depth 4. RandomizedSearchCV found a slightly higher forest score (0.733 vs 0.724) with 20% fewer model fits, but the gap is far below the fold-to-fold spread, so it is not a real difference. The validation curve shows that deeper forests keep improving on the training data (to 1.00) while the validation score stays flat at about 0.72: more complexity brings no benefit, only overfitting.

Step 8: Final evaluation on the test set

The tuned models are scored once on the untouched test set (103 applicants). The **confusion matrix** counts the four outcomes: true positives (TP, approved and predicted approved), true negatives (TN), false positives (FP, rejected but predicted approved) and false negatives (FN, approved but predicted rejected). **Precision** = TP / (TP + FP) answers 'of those predicted approved, how many really were?'. **Recall** = TP / (TP + FN) answers 'of those really approved, how many did we find?'. **F1** is their harmonic mean. **ROC-AUC** measures how well the model ranks approved above rejected applicants at every threshold. The best model is chosen by cross-validated AUC, not by the test score.

CODE

```
final = {"Baseline (always approve)": models["Baseline (always approve)"].fit(X_train, y_train), **tuned}
rows, preds, probas = [], {}, {}
for name, model in final.items():
    preds[name] = model.predict(X_test)
    probas[name] = model.predict_proba(X_test)[:, 1]
    rows.append({"Model": name,
                 "Accuracy": accuracy_score(y_test, preds[name]),
                 "Precision": precision_score(y_test, preds[name], zero_division=0),
                 "Recall": recall_score(y_test, preds[name]),
                 "F1": f1_score(y_test, preds[name]),
                 "ROC_AUC": roc_auc_score(y_test, probas[name])})
test_metrics = pd.DataFrame(rows).set_index("Model")
print(test_metrics.round(3))
best = tuned_table["CV_AUC"].idxmax()                  # chosen by cross-validation, not by the test set
print(f"\nBest model (highest cross-validated AUC): {best}")
tn, fp, fn, tp = confusion_matrix(y_test, preds[best]).ravel()
print(f"\nConfusion matrix of {best}:")
print(pd.DataFrame([[tn, fp], [fn, tp]], index=["Actual Rejected", "Actual Approved"],
                   columns=["Pred. Rejected", "Pred. Approved"]))
print(f"\nTN={tn}  FP={fp}  FN={fn}  TP={tp}")
print(f"Precision = TP / (TP + FP) = {tp}/{tp + fp} = {tp / (tp + fp):.3f}")
print(f"Recall    = TP / (TP + FN) = {tp}/{tp + fn} = {tp / (tp + fn):.3f}")
print(f"Specificity = TN / (TN + FP) = {tn}/{tn + fp} = {tn / (tn + fp):.3f}")
print("\nClassification report:")
print(classification_report(y_test, preds[best], target_names=["Rejected", "Approved"], digits=3))
tuned_only = {k: v for k, v in preds.items() if k in tuned}
plot_confusion(tuned_only, y_test)
plot_roc({k: probas[k] for k in tuned}, y_test, test_metrics["ROC_AUC"])
test_metrics.round(4).to_csv(f"{OUT}/test_metrics.csv")
joblib.dump(final[best], "models/best_classifier.joblib")
```

OUTPUT

```
                          Accuracy  Precision  Recall     F1  ROC_AUC
Model
Baseline (always approve)     0.612      0.612   1.000  0.759    0.500
Logistic Regression           0.718      0.736   0.841  0.785    0.756
KNN                           0.650      0.671   0.841  0.746    0.749
Random Forest                 0.709      0.714   0.873  0.786    0.755
```

```
Best model (highest cross-validated AUC): Logistic Regression

Confusion matrix of Logistic Regression:
                 Pred. Rejected  Pred. Approved
Actual Rejected              21              19
Actual Approved              10              53

TN=21  FP=19  FN=10  TP=53
Precision = TP / (TP + FP) = 53/72 = 0.736
Recall    = TP / (TP + FN) = 53/63 = 0.841
Specificity = TN / (TN + FP) = 21/40 = 0.525

Classification report:
              precision    recall  f1-score   support

    Rejected      0.677     0.525     0.592        40
    Approved      0.736     0.841     0.785        63

    accuracy                          0.718       103
   macro avg      0.707     0.683     0.688       103
weighted avg      0.713     0.718     0.710       103
```

Test-set results of the tuned models

Model	Accuracy	Precision	Recall	F1	ROC-AUC
Baseline (always approve)	0.612	0.612	1.000	0.759	0.500
Logistic Regression	0.718	0.736	0.841	0.785	0.756
KNN	0.650	0.671	0.841	0.746	0.749
Random Forest	0.709	0.714	0.873	0.786	0.755

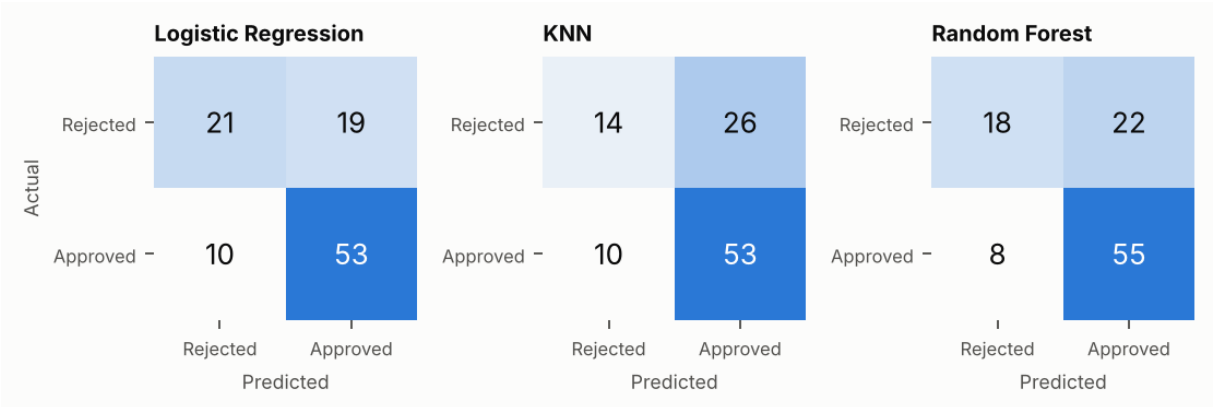


Figure 8 - Confusion matrices of the tuned models (rows = actual, columns = predicted).

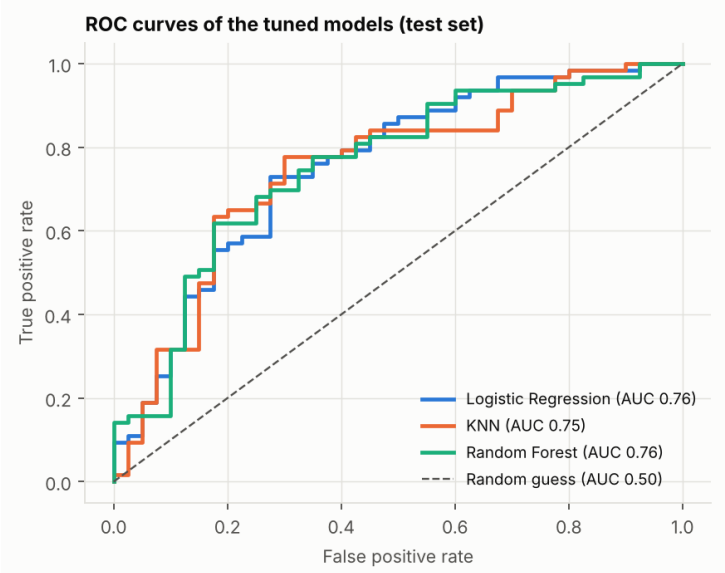


Figure 9 - ROC curves: the closer to the top-left corner, the better.

Result: Best model: Logistic Regression (accuracy 0.718, precision 0.736, recall 0.841, F1 0.785, ROC-AUC 0.756). Its confusion matrix has TP = 53, TN = 21, FP = 19, FN = 10. It finds most approved applicants (recall 0.84) but is weak at spotting rejections: only 21 of 40 rejected applicants were caught (specificity 0.525). The baseline scores accuracy 0.612 and AUC 0.500, so the models add real but modest skill (+0.11 accuracy, +0.26 AUC). The three tuned models are statistically indistinguishable on 103 test rows.

Step 9: Decision threshold - the precision / recall trade-off

A classifier outputs a probability, and the **decision threshold** turns it into a yes/no answer. The default is 0.5, but it can be moved depending on which mistake is more costly. A higher threshold approves fewer applicants: precision rises and recall falls. A lower threshold does the opposite.

CODE

```
p = probas[best]
rows = []
for t in [0.3, 0.4, 0.5, 0.6, 0.7, 0.8]:
    pr = (p >= t).astype(int)                       # predict "approved" only if probability >= t
    rows.append({"Threshold": t, "Precision": precision_score(y_test, pr, zero_division=0),
                 "Recall": recall_score(y_test, pr), "F1": f1_score(y_test, pr),
                 "Accuracy": accuracy_score(y_test, pr)})
thr_table = pd.DataFrame(rows).set_index("Threshold")
print(f"Effect of the decision threshold for {best}:")
print(thr_table.round(3))
thr = np.linspace(0.2, 0.9, 36)
pr_c = [precision_score(y_test, (p >= t).astype(int), zero_division=0) for t in thr]
re_c = [recall_score(y_test, (p >= t).astype(int)) for t in thr]
f1_c = [f1_score(y_test, (p >= t).astype(int)) for t in thr]
plot_threshold(thr, pr_c, re_c, f1_c)
thr_table.round(4).to_csv(f"{OUT}/threshold_analysis.csv")
```

OUTPUT

```
Effect of the decision threshold for Logistic Regression:
           Precision  Recall     F1  Accuracy
Threshold
0.3            0.653   0.984  0.785     0.670
0.4            0.686   0.937  0.792     0.699
0.5            0.736   0.841  0.785     0.718
0.6            0.780   0.730  0.754     0.709
0.7            0.844   0.429  0.568     0.602
0.8            0.824   0.222  0.350     0.495
```

Effect of the decision threshold

Threshold	Precision	Recall	F1	Accuracy
0.3	0.653	0.984	0.785	0.670
0.4	0.686	0.937	0.792	0.699
0.5	0.736	0.841	0.785	0.718
0.6	0.780	0.730	0.754	0.709
0.7	0.844	0.429	0.568	0.602
0.8	0.824	0.222	0.350	0.495

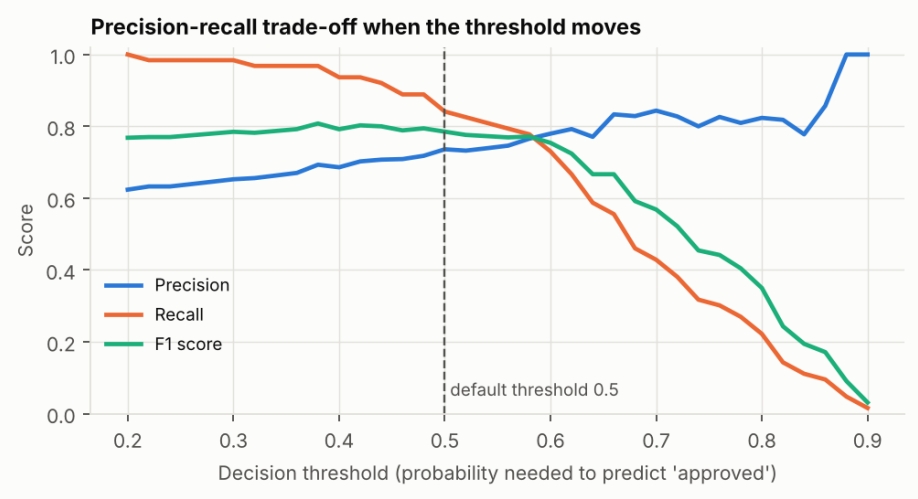


Figure 10 - Precision, recall and F1 as the decision threshold moves.

Result: At threshold 0.7 precision rises to 0.844 but recall collapses to 0.429. At 0.4 recall reaches 0.937 and F1 peaks at 0.792. For a lender who mostly fears bad loans (false positives) a threshold around 0.6 is safer; for a lender who does not want to lose good customers a threshold around 0.4 is better. The right value is a business decision, not a statistical one.

Part C - Why do the models perform differently?

Step 10: Feature importance - do the three models rely on the same features?

Feature importance answers the question 'which columns does a model actually use?'. Three views are compared. **Logistic Regression coefficients** show the direction and size of each feature's effect (numeric columns are scaled, so a coefficient is the change in log-odds per standard deviation; the 0/1 dummy columns are not scaled, so their coefficients are not directly comparable). **Random Forest impurity importance** adds up how much each feature reduces impurity across all trees; it is known to favour columns with many distinct values. **Permutation importance** shuffles one column of the test set and measures how much the ROC-AUC drops. It works the same way for every model, including KNN, which has no built-in importance, and it measures what the model really relies on. The shuffle is repeated 30 times and its spread (noise) shows which differences are real. **Spearman correlation** measures how much two rankings agree (1 = same order).

CODE

```
fitted = {"Logistic Regression": tuned["Logistic Regression"], "KNN": tuned["KNN"],
          "Random Forest": tuned["Random Forest"]}
cols = list(X.columns)

# 10a. Built-in importance, read from inside the trained models (training data)
coef = pd.Series(fitted["Logistic Regression"].coef_[0], index=cols)
rf_imp = pd.Series(fitted["Random Forest"].feature_importances_, index=cols)
built_in = pd.DataFrame({"LR_coefficient": coef, "LR_rank": coef.abs().rank(ascending=False).astype(int),
                         "RF_importance": rf_imp, "RF_rank": rf_imp.rank(ascending=False).astype(int)})
print("Built-in importance (LR: coefficient per 1 std for numeric columns, per 0->1 for dummies;")
print("RF: share of the total impurity reduction):")
print(built_in.sort_values("RF_importance", ascending=False).round(3))

# 10b. Permutation importance: shuffle one column on the test set and measure how much the AUC drops.
#      Works the same way for every model, including KNN which has no built-in importance.
perm = {}
for name, model in fitted.items():
    r = permutation_importance(model, X_test, y_test, scoring="roc_auc", n_repeats=30, random_state=42)
    perm[name] = pd.DataFrame({"mean": r.importances_mean, "std": r.importances_std}, index=cols)
perm_mean = pd.DataFrame({n: d["mean"] for n, d in perm.items()})
perm_mean["Average"] = perm_mean.mean(axis=1)
print("\nPermutation importance = drop in test ROC-AUC (mean of 30 shuffles):")
print(perm_mean.sort_values("Average", ascending=False).round(3))
perm_noise = pd.Series({n: d["std"].median() for n, d in perm.items()})
print("\nTypical shuffle-to-shuffle noise (median std over the 14 features):")
print(perm_noise.round(3).to_string())
print("Differences smaller than about twice this noise are not reliable.")

# 10c. Do the models agree on the ranking? (Spearman rank correlation: 1 = same order)
agree = pd.DataFrame({**{f"Perm. {n}": perm_mean[n] for n in fitted},
                      "LR |coefficient|": coef.abs(), "RF impurity": rf_imp}).corr(method="spearman")
print("\nAgreement between the feature rankings (Spearman correlation):")
print(agree.round(2))
```

```
plot_perm_importance(perm)
perm_mean.round(4).to_csv(f"{OUT}/permutation_importance.csv")
built_in.round(4).to_csv(f"{OUT}/builtin_importance.csv")
```

OUTPUT

```
Built-in importance (LR: coefficient per 1 std for numeric columns, per 0->1 for dummies;
RF: share of the total impurity reduction):
                               LR_coefficient  LR_rank  RF_importance  RF_rank
Credit_Score                            0.869        1          0.405        1
Loan_Amount                            -0.331        3          0.159        2
Annual_Income                           0.224        4          0.131        3
Age                                     0.011       13          0.097        4
Loan_Term_Months                       -0.030       12          0.046        5
Education                               0.006       14          0.039        6
Gender                                 -0.184        5          0.022        7
Employment_Status_Unemployed           -0.337        2          0.021        8
City_Delhi                             -0.139        9          0.016        9
City_Mumbai                             0.103       10          0.016       10
Employment_Status_Salaried             -0.159        7          0.015       11
City_Lucknow                           -0.163        6          0.014       12
City_Kanpur                             0.154        8          0.011       13
Employment_Status_Self-Employed         0.076       11          0.008       14

Permutation importance = drop in test ROC-AUC (mean of 30 shuffles):
                               Logistic Regression    KNN  Random Forest  Average
Credit_Score                                 0.221  0.179          0.199    0.200
Annual_Income                                0.023  0.001          0.004    0.009
Age                                          0.002  0.013          0.011    0.008
Employment_Status_Unemployed                 0.005  0.001          0.009    0.005
City_Kanpur                                  0.007  0.001          0.002    0.004
City_Delhi                                   0.004  0.005          0.000    0.003
Loan_Amount                                  0.010 -0.001         -0.001    0.003
City_Mumbai                                 -0.003  0.005          0.006    0.002
Gender                                       0.004 -0.001          0.003    0.002
City_Lucknow                                 0.005 -0.004          0.000    0.000
Education                                    0.000 -0.005          0.003   -0.000
Employment_Status_Self-Employed             -0.002  0.002         -0.006   -0.002
Employment_Status_Salaried                   0.001 -0.000         -0.007   -0.002
Loan_Term_Months                            -0.000 -0.004         -0.010   -0.005

Typical shuffle-to-shuffle noise (median std over the 14 features):
Logistic Regression    0.005
KNN                    0.013
Random Forest          0.004
Differences smaller than about twice this noise are not reliable.

Agreement between the feature rankings (Spearman correlation):
                          Perm. Logistic Regression  Perm. KNN  Perm. Random Forest  LR |coefficient|  \
Perm. Logistic Regression                       1.00       0.16                0.34              0.79
Perm. KNN                                       0.16       1.00                0.53              0.10
Perm. Random Forest                             0.34       0.53                1.00              0.24
LR |coefficient|                                0.79       0.10                0.24              1.00
RF impurity                                     0.45       0.02                0.41              0.29

                          RF impurity
Perm. Logistic Regression         0.45
Perm. KNN                         0.02
Perm. Random Forest               0.41
LR |coefficient|                  0.29
RF impurity                       1.00
```

Permutation importance (drop in test ROC-AUC) of the 8 most important features

Feature	Logistic Regression	KNN	Random Forest	Average
Credit_Score	0.221	0.179	0.199	0.200
Annual_Income	0.023	0.001	0.004	0.009
Age	0.002	0.013	0.011	0.008
Employment_Status_Unemployed	0.005	0.001	0.009	0.005
City_Kanpur	0.007	0.001	0.002	0.004
City_Delhi	0.004	0.005	0.000	0.003
Loan_Amount	0.010	-0.001	-0.001	0.003
City_Mumbai	-0.003	0.005	0.006	0.002

Built-in importance (Logistic Regression coefficient, Random Forest impurity)

Feature	LR coefficient	LR rank	RF importance	RF rank
Credit_Score	+0.869	1	0.405	1
Loan_Amount	-0.331	3	0.159	2
Annual_Income	+0.224	4	0.131	3
Age	+0.011	13	0.097	4
Loan_Term_Months	-0.030	12	0.046	5
Education	+0.006	14	0.039	6
Gender	-0.184	5	0.022	7
Employment_Status_Unemployed	-0.337	2	0.021	8

Agreement between the importance rankings (Spearman correlation)

	Perm: LR	Perm: KNN	Perm: RF	LR |coefficient|	RF impurity
Perm: LR	1.00	0.16	0.34	0.79	0.45
Perm: KNN	0.16	1.00	0.53	0.10	0.02
Perm: RF	0.34	0.53	1.00	0.24	0.41
LR |coefficient|	0.79	0.10	0.24	1.00	0.29
RF impurity	0.45	0.02	0.41	0.29	1.00

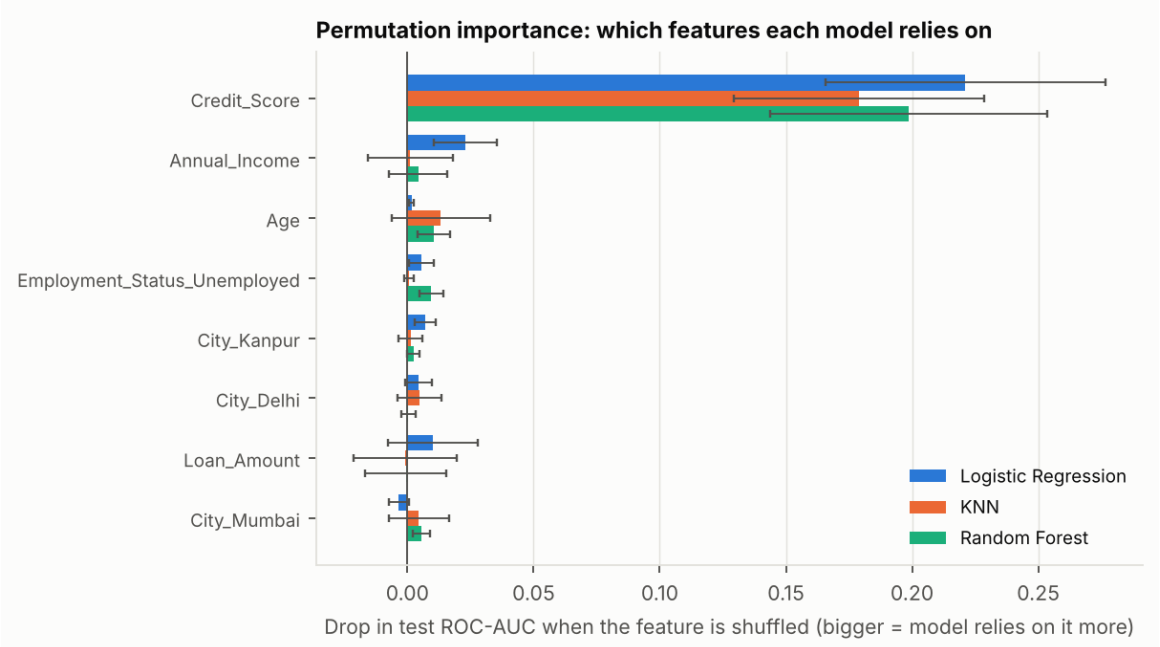


Figure 11 - Permutation importance of the eight most important features for each model. The lines show the spread over 30 shuffles.

Result: Every model and every method puts credit score first. Shuffling it drops the test AUC by 0.221 (Logistic Regression), 0.179 (KNN) and 0.199 (Random Forest), which takes each model from about 0.75 down to 0.53-0.57, close to random guessing. **Below credit score the models disagree.** Logistic Regression uses Annual_Income (0.023) and Loan_Amount (0.010) together, with opposite-sign coefficients (-0.33 and +0.22): a big loan relative to income lowers approval. KNN and Random Forest rely on Age instead (0.013 and 0.011), probably as a stand-in for income (age and income are correlated, r = 0.74 in Week 1). The shuffle noise is only about 0.005 (Logistic Regression), 0.013 (KNN) and 0.004 (Random Forest), so apart from credit score most differences are small. Rank agreement between models is low (Spearman 0.16 LR-KNN, 0.34 LR-RF, 0.53 KNN-RF). **Two warnings.** Impurity importance ranks Loan_Term_Months (0.046) and Education (0.039) 5th and 6th although shuffling them does not lower the AUC, so it overstates them. And the 'Unemployed' dummy has the second-largest regression coefficient (-0.34) but almost no effect on AUC (0.005), because it describes only about 5% of applicants: a large coefficient is not the same as an important feature.

Step 11: Why do the models perform differently? Four experiments

Four experiments look for the reasons behind the ranking of the models. **(a) Feature ablation:** the same tuned models are cross-validated on smaller feature sets, and on a set with an **engineered feature**, loan-to-income (loan amount divided by income, a standard lending ratio), to see which model is hurt by useless columns or helped by better inputs. **(b) Learning curves** train on growing parts of the training data: a large gap between training and validation score means high variance (the model memorises), and a validation score that is still rising means more data would help. **(c) A corrected paired t-test** on the same 15 folds asks whether the differences between the models are real; the Nadeau-Bengio correction is used because the folds overlap and a plain t-test would be too optimistic. **(d) Noise ceiling:** because the dataset was generated in Week 1, the rule that produced the approval label is known. Applying it to the cleaned data gives an approximate upper limit for any model. This is only possible for synthetic data.

CODE

```
SC = SCALE
credit_raw = X["Credit_Score"] * SC["Credit_Score"][1] + SC["Credit_Score"][0]
income_raw = X["Annual_Income"] * SC["Annual_Income"][1] + SC["Annual_Income"][0]
loan_raw = X["Loan_Amount"] * SC["Loan_Amount"][1] + SC["Loan_Amount"][0]
ratio_raw = loan_raw / income_raw                              # loan-to-income: a standard lending ratio
r_tr = ratio_raw.loc[X_train.index]
X_fe = X.assign(Loan_to_Income=(ratio_raw - r_tr.mean()) / r_tr.std())   # scaled with training statistics
X_train_fe, X_test_fe = X_fe.loc[X_train.index], X_fe.loc[X_test.index]

# 11a. Feature ablation: the same tuned models on smaller and on engineered feature sets
financial = ["Credit_Score", "Annual_Income", "Loan_Amount"]
sets = {
    "Credit score only": ["Credit_Score"],
    "3 financial features": financial,
    "6 numeric features": ["Age", "Education", "Annual_Income", "Credit_Score", "Loan_Amount", "Loan_Term_Months"],
    "All 14 features": cols,
    "Credit score + loan-to-income": ["Credit_Score", "Loan_to_Income"],
    "All 14 + loan-to-income": cols + ["Loan_to_Income"],
}
ablation = pd.DataFrame({
    sname: {m: cross_val_score(clone(model), X_train_fe[fs], y_train, cv=rcv, scoring="roc_auc").mean()
            for m, model in fitted.items()} for sname, fs in sets.items()}).T
print("11a. Cross-validated ROC-AUC (15 folds) for each feature set:")
print(ablation.round(3))
plot_ablation(ablation)
ablation.round(4).to_csv(f"{OUT}/feature_ablation.csv")

# 11b. Learning curves: does more training data help?
lc, rows = {}, []
for name, model in fitted.items():
    n_tr, tr, va = learning_curve(clone(model), X_train, y_train, train_sizes=[0.2, 0.4, 0.6, 0.8, 1.0], cv=skf,
                                  scoring="roc_auc", shuffle=True, random_state=42)
    lc[name] = (n_tr, tr, va)
    for n, a, b in zip(n_tr, tr.mean(axis=1), va.mean(axis=1)):
        rows.append({"Model": name, "Train_rows": int(n), "Train_AUC": a, "Validation_AUC": b, "Gap": a - b})
print("\n11b. Learning curves (5-fold CV at growing training sizes):")
print(pd.DataFrame(rows).set_index(["Model", "Train_rows"]).round(3))
plot_learning(lc)

# 11c. Are the differences between the models real? Corrected paired t-test on the 15 CV folds
#      (Nadeau-Bengio correction: folds overlap, so the plain t-test is too optimistic).
fold_auc = {m: cross_val_score(clone(model), X_train, y_train, cv=rcv, scoring="roc_auc")
            for m, model in fitted.items()}
pairs, rows = [("Logistic Regression", "KNN"), ("Logistic Regression", "Random Forest"),
               ("Random Forest", "KNN")], []
for a, b in pairs:
    d = fold_auc[a] - fold_auc[b]
    se = np.sqrt((1 / len(d) + 0.25) * d.var(ddof=1))          # test/train size ratio is 1/4 for 5 folds
    tstat = d.mean() / se
    rows.append({"Comparison": f"{a} - {b}", "Mean_AUC_diff": d.mean(), "t": tstat,
                 "p_value": 2 * t_dist.sf(abs(tstat), len(d) - 1), "Folds_A_better": int((d > 0).sum())})
paired = pd.DataFrame(rows).set_index("Comparison")
print("\n11c. Paired comparison of the tuned models (same 15 folds):")
print(paired.round(3))

# 11d. Noise ceiling. Because the dataset was generated in Week 1, the rule that produced the label is known.
#      Applying it to the cleaned data gives an approximate upper limit for ANY model.
unemp = X["Employment_Status_Unemployed"]
true_logit = (0.8 * (credit_raw - 600) / 50 + 0.5 * (income_raw - 650000) / 300000
              - 1.2 * (loan_raw / income_raw - 0.9) - 1.5 * unemp)
p_true = 1 / (1 + np.exp(-true_logit))
ceiling = {"AUC": roc_auc_score(y, p_true), "Accuracy": accuracy_score(y, (p_true >= 0.5).astype(int)),
           "AUC_test_rows": roc_auc_score(y_test, p_true.loc[y_test.index])}
print("\n11d. Approximate ceiling (true generating rule applied to the cleaned data):")
print(f"AUC on all 515 rows = {ceiling['AUC']:.3f}, accuracy = {ceiling['Accuracy']:.3f}, "
      f"AUC on the 103 test rows = {ceiling['AUC_test_rows']:.3f}")
```

```
print(f"Best model on the test rows ({best}): AUC = {test_metrics.loc[best, 'ROC_AUC']:.3f}")
print("(The label was drawn at random from these probabilities, so even the true rule cannot be perfect.)")
```

OUTPUT

```
11a. Cross-validated ROC-AUC (15 folds) for each feature set:
                           Logistic Regression    KNN  Random Forest
Credit score only                        0.731  0.661          0.716
3 financial features                     0.742  0.713          0.736
6 numeric features                       0.736  0.710          0.726
All 14 features                          0.734  0.698          0.718
Credit score + loan-to-income            0.746  0.675          0.722
All 14 + loan-to-income                  0.737  0.703          0.719

11b. Learning curves (5-fold CV at growing training sizes):
                             Train_AUC  Validation_AUC    Gap
Model               Train_rows
Logistic Regression 65             0.801           0.720  0.081
                    131            0.793           0.744  0.049
                    197            0.773           0.741  0.032
                    263            0.775           0.742  0.033
                    329            0.768           0.737  0.030
KNN                 65             1.000           0.674  0.326
                    131            1.000           0.692  0.308
                    197            1.000           0.710  0.290
                    263            1.000           0.707  0.293
                    329            1.000           0.716  0.284
Random Forest       65             0.936           0.676  0.260
                    131            0.938           0.713  0.225
                    197            0.918           0.710  0.208
                    263            0.903           0.726  0.177
                    329            0.890           0.728  0.162

11c. Paired comparison of the tuned models (same 15 folds):
                                    Mean_AUC_diff      t  p_value  Folds_A_better
Comparison
Logistic Regression - KNN                   0.037  1.228    0.240              11
Logistic Regression - Random Forest         0.016  1.248    0.233              11
Random Forest - KNN                         0.020  0.774    0.452               9

11d. Approximate ceiling (true generating rule applied to the cleaned data):
AUC on all 515 rows = 0.744, accuracy = 0.697, AUC on the 103 test rows = 0.759
Best model on the test rows (Logistic Regression): AUC = 0.756
(The label was drawn at random from these probabilities, so even the true rule cannot be perfect.)
```

Cross-validated ROC-AUC for each feature set (11a)

Feature set	Logistic Regression	KNN	Random Forest
Credit score only	0.731	0.661	0.716
3 financial features	0.742	0.713	0.736
6 numeric features	0.736	0.710	0.726
All 14 features	0.734	0.698	0.718
Credit score + loan-to-income	0.746	0.675	0.722
All 14 + loan-to-income	0.737	0.703	0.719

Learning curves: ROC-AUC at growing training sizes (11b)

Model	Score	65 rows	131 rows	197 rows	263 rows	329 rows
Logistic Regression	Training AUC	0.801	0.793	0.773	0.775	0.768
	Validation AUC	0.720	0.744	0.741	0.742	0.737
KNN	Training AUC	1.000	1.000	1.000	1.000	1.000
	Validation AUC	0.674	0.692	0.710	0.707	0.716
Random Forest	Training AUC	0.936	0.938	0.918	0.903	0.890
	Validation AUC	0.676	0.713	0.710	0.726	0.728

Corrected paired comparison on the same 15 folds (11c)

Comparison	Mean AUC difference	Folds where first is better	Corrected t	p-value
Logistic Regression - KNN	+0.037	11 of 15	1.23	0.24
Logistic Regression - Random Forest	+0.016	11 of 15	1.25	0.23
Random Forest - KNN	+0.020	9 of 15	0.77	0.45

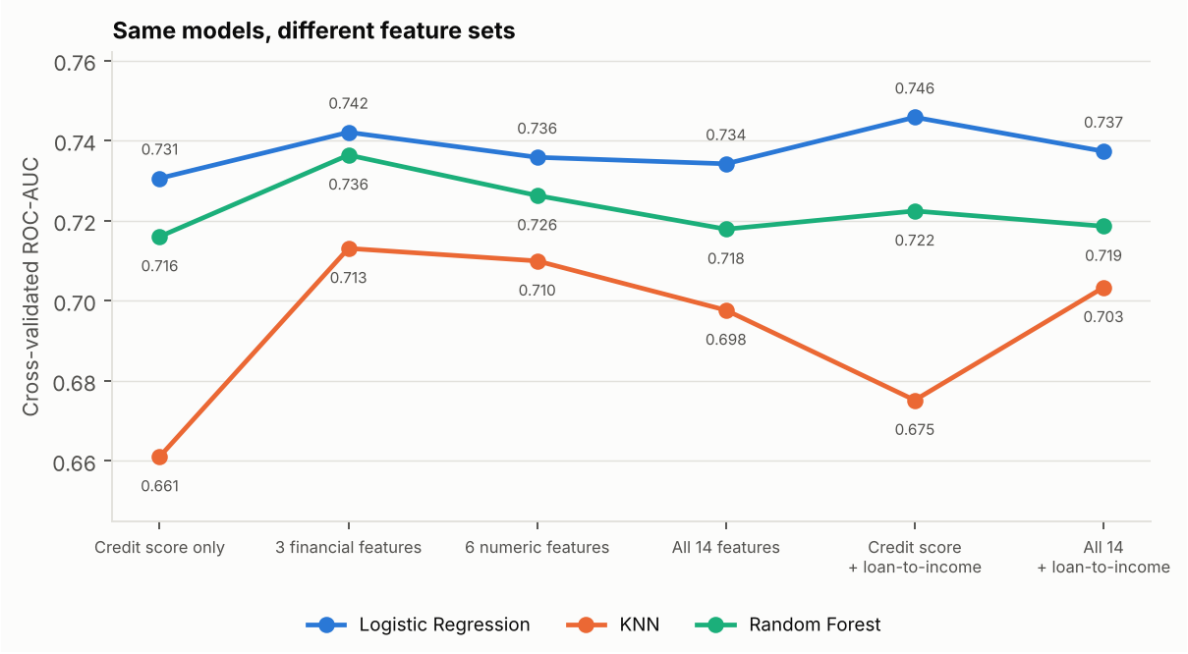


Figure 12 - Cross-validated ROC-AUC of each model on six feature sets.

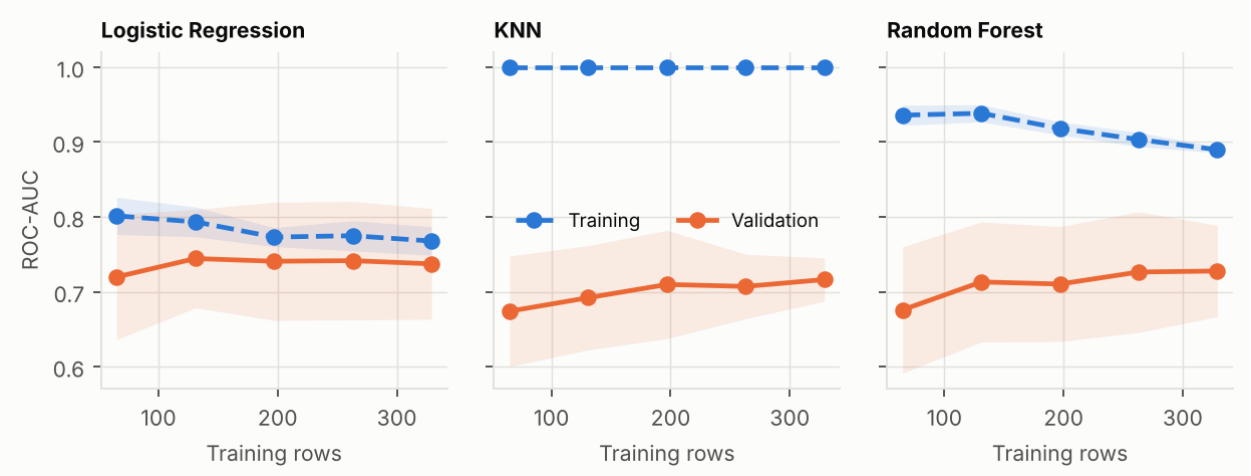


Figure 13 - Learning curves: training score (dashed) and validation score (solid) at growing training sizes.

Result: (a) Logistic Regression is steady (0.731-0.746 whatever the features). KNN is hurt most by extra columns (0.713 with 3 financial features vs 0.698 with all 14) and Random Forest also gains from fewer columns (0.736 vs 0.718). Adding the loan-to-income ratio to credit score lifts every model above credit score alone (Logistic Regression +0.015, KNN +0.014, Random Forest +0.006), but these gains are smaller than the fold-to-fold spread. **(b)** For Logistic Regression the training and validation scores meet (gap 0.03 at full size): the model is stable and more data will not help. KNN's training AUC is 1.00 at every size, because with distance weighting each training point is its own nearest neighbour, so its gap is 0.28. The Random Forest gap shrinks from 0.26 to 0.16 and its validation AUC is still rising (0.676 to 0.728): it is the model that would gain most from more data. **(c)** Logistic Regression beats Random Forest by +0.016 AUC and KNN by +0.037 on average (better in 11 of 15 folds), but the corrected p-values are 0.23 and 0.24: **the ranking of the models is not statistically established**. **(d)** The true rule gives an AUC of 0.744 (accuracy 0.697) on all 515 rows and 0.759 on the test rows. The best model reaches 0.739 (accuracy 0.699) out-of-fold and 0.756 on the test rows, so it is already at the approximate ceiling (the AUC estimate itself has an uncertainty of about 0.02).

Step 12: Error analysis - which applicants do the models get wrong?

Averages hide where a model fails. To use all 515 applicants, **out-of-fold predictions** are made: each applicant is predicted by a model that never saw them (5-fold). Errors are then grouped by credit-score band, loan-to-income band and employment status. A benchmark is added: inside any band, the **best constant guess** (always predict the more common outcome of that band) makes errors equal to the smaller share. A model only adds value if it beats this benchmark. Finally, the applicants that all three models get wrong are compared with those that all get right.

CODE

```
oof = {m: cross_val_predict(clone(model), X, y, cv=skf, method="predict_proba")[:, 1] for m, model in fitted.items()}
pred = {m: (p >= 0.5).astype(int) for m, p in oof.items()}      # out-of-fold: every row predicted by a model that
    never saw it
oof_tab = pd.DataFrame({m: {"AUC": roc_auc_score(y, oof[m]), "Accuracy": accuracy_score(y, pred[m]),
                            "Precision": precision_score(y, pred[m]), "Recall": recall_score(y, pred[m])}
                        for m in fitted}).T
print("Out-of-fold results on all 515 applicants (a larger sample than the 103-row test set):")
print(oof_tab.round(3))

wrong = pd.DataFrame({m: pred[m] != y.to_numpy() for m in fitted}, index=X.index)
n_wrong = wrong.sum(axis=1)
print("\nHow many of the 3 models get an applicant wrong:")
print(n_wrong.value_counts().sort_index().rename("Applicants").to_frame().assign(
    Share_pct=lambda d: (d["Applicants"] / len(X) * 100).round(1)))
print("\nPairwise disagreement (share of applicants where two models give different answers):")
names = list(fitted)
print(pd.DataFrame([[np.mean(pred[a] != pred[b]) for b in names] for a in names], index=names,
    columns=names).round(3))

emp = pd.Series(np.select([X["Employment_Status_Salaried"] == 1, X["Employment_Status_Self-Employed"] == 1,
                           unemp == 1], ["Salaried", "Self-Employed", "Unemployed"], "Business"), index=X.index)
credit_q = pd.qcut(credit_raw, 4, labels=["Q1 lowest", "Q2", "Q3", "Q4 highest"])
lti_q = pd.qcut(ratio_raw, 4, labels=["Q1 lowest", "Q2", "Q3", "Q4 highest"])
err = lambda g: wrong.groupby(g, observed=True).mean()
err_credit, err_lti, err_emp = err(credit_q), err(lti_q), err(emp)
print("\nError rate by credit-score quartile (Q1 = lowest scores):")
print(err_credit.round(3))
print("\nError rate by loan-to-income quartile (Q1 = smallest loan relative to income):")
print(err_lti.round(3))
print("\nError rate by employment status:")
print(err_emp.round(3))
print("\nApproval rate actually observed in each credit-score quartile (%):")
print((y.groupby(credit_q, observed=True).mean() * 100).round(1))
# Benchmark: inside a band, the best a model could do with NO extra information is to guess the majority outcome
best_guess = lambda g: y.groupby(g, observed=True).mean().apply(lambda p: min(p, 1 - p))
base_credit, base_lti = best_guess(credit_q), best_guess(lti_q)
print("\nError of the best constant guess inside each band (what a model must beat to add value):")
print(pd.DataFrame({"Credit_quartile": base_credit.to_numpy(), "LTI_quartile": base_lti.to_numpy()},
                   index=["Q1 lowest", "Q2", "Q3", "Q4 highest"]).round(3))

lr_pred = pred["Logistic Regression"]
fp_share = np.mean((lr_pred == 1) & (y.to_numpy() == 0))
fn_share = np.mean((lr_pred == 0) & (y.to_numpy() == 1))
print(f"\nLogistic Regression mistakes: wrongly approved {fp_share * 100:.1f}% of applicants, "
      f"wrongly rejected {fn_share * 100:.1f}%")
hard = n_wrong == 3
profile_hard = pd.DataFrame({"Wrong_for_all_3": [credit_raw[hard].mean(), ratio_raw[hard].mean(),
    income_raw[hard].mean(),
                                                 y[hard].mean() * 100],
                             "Correct_for_all_3": [credit_raw[n_wrong == 0].mean(), ratio_raw[n_wrong == 0].mean(),
                                                   income_raw[n_wrong == 0].mean(), y[n_wrong == 0].mean() * 100]},
                            index=["Credit score", "Loan-to-income", "Income (Rs.)", "Actually approved (%)"])
print(f"\nProfile of the {int(hard.sum())} applicants that all three models get wrong vs those all get right:")
print(profile_hard.round(2))
plot_errors(err_credit, err_lti, base_credit, base_lti)
oof_tab.round(4).to_csv(f"{OUT}/out_of_fold_results.csv")
```

OUTPUT

```
Out-of-fold results on all 515 applicants (a larger sample than the 103-row test set):
                     AUC  Accuracy  Precision  Recall
Logistic Regression  0.739     0.699      0.714   0.851
KNN                  0.708     0.652      0.669   0.858
Random Forest        0.728     0.685      0.693   0.873

How many of the 3 models get an applicant wrong:
   Applicants  Share_pct
0         308       59.8
1          50        9.7
2          25        4.9
3         132       25.6

Pairwise disagreement (share of applicants where two models give different answers):
                     Logistic Regression    KNN  Random Forest
```

```
Logistic Regression               0.000  0.117          0.072
KNN                               0.117  0.000          0.103
Random Forest                     0.072  0.103          0.000

Error rate by credit-score quartile (Q1 = lowest scores):
              Logistic Regression    KNN  Random Forest
Credit_Score
Q1 lowest                   0.357  0.450          0.395
Q2                          0.396  0.472          0.403
Q3                          0.276  0.284          0.284
Q4 highest                  0.159  0.159          0.159

Error rate by loan-to-income quartile (Q1 = smallest loan relative to income):
            Logistic Regression    KNN  Random Forest
Q1 lowest                 0.326  0.357          0.341
Q2                        0.318  0.349          0.341
Q3                        0.242  0.289          0.250
Q4 highest                0.318  0.395          0.326

Error rate by employment status:
               Logistic Regression    KNN  Random Forest
Business                     0.286  0.306          0.286
Salaried                     0.314  0.346          0.327
Self-Employed                0.247  0.358          0.284
Unemployed                   0.370  0.481          0.370

Approval rate actually observed in each credit-score quartile (%):
Credit_Score
Q1 lowest     31.8
Q2            59.7
Q3            71.6
Q4 highest    84.1
Name: Approved, dtype: float64

Error of the best constant guess inside each band (what a model must beat to add value):
            Credit_quartile  LTI_quartile
Q1 lowest             0.318         0.310
Q2                    0.403         0.372
Q3                    0.284         0.414
Q4 highest            0.159         0.450

Logistic Regression mistakes: wrongly approved 21.0% of applicants, wrongly rejected 9.1%

Profile of the 132 applicants that all three models get wrong vs those all get right:
                     Wrong_for_all_3  Correct_for_all_3
Credit score                601.58             621.47
Loan-to-income                0.87               0.88
Income (Rs.)             767042.09          803109.79
Actually approved (%)        21.21              81.49
```

Out-of-fold results on all 515 applicants

Model	AUC	Accuracy	Precision	Recall
Logistic Regression	0.739	0.699	0.714	0.851
KNN	0.708	0.652	0.669	0.858
Random Forest	0.728	0.685	0.693	0.873

Error rate by credit-score quartile

Credit-score quartile	Actually approved	Best constant guess	LR error	KNN error	RF error
Q1 lowest	31.8%	31.8%	35.7%	45.0%	39.5%
Q2	59.7%	40.3%	39.6%	47.2%	40.3%
Q3	71.6%	28.4%	27.6%	28.4%	28.4%
Q4 highest	84.1%	15.9%	15.9%	15.9%	15.9%

Error rate by loan-to-income quartile

Loan-to-income quartile	Actually approved	Best constant guess	LR error	KNN error	RF error
Q1 lowest	69.0%	31.0%	32.6%	35.7%	34.1%
Q2	62.8%	37.2%	31.8%	34.9%	34.1%
Q3	58.6%	41.4%	24.2%	28.9%	25.0%
Q4 highest	55.0%	45.0%	31.8%	39.5%	32.6%

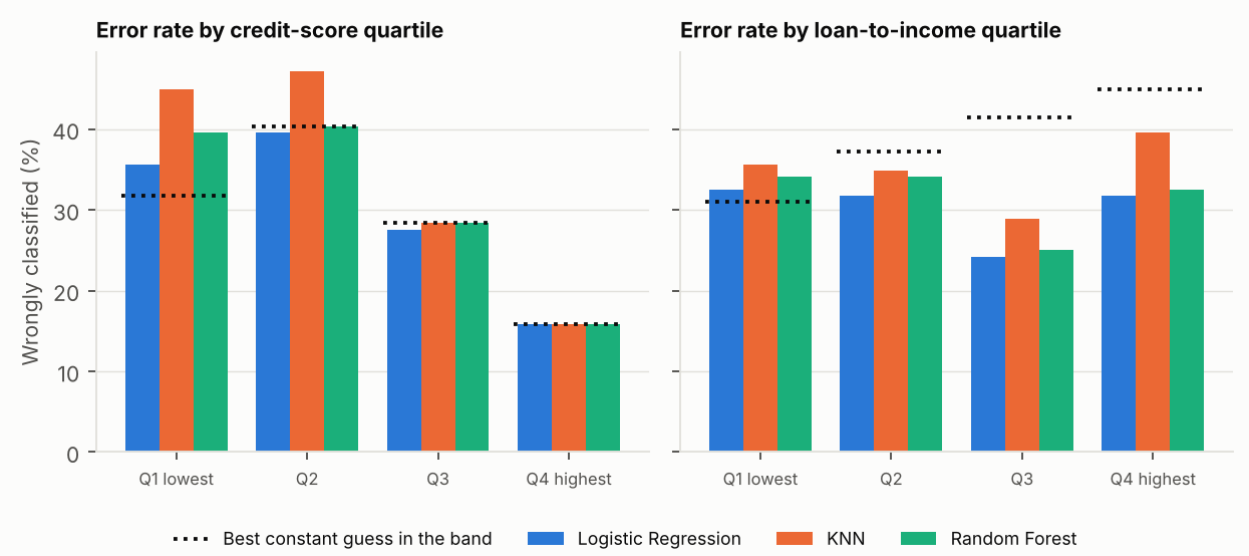


Figure 14 - Error rate of each model inside credit-score and loan-to-income bands. The dotted line is the error of the best constant guess in the band.

Result: Out-of-fold AUC is 0.739 (Logistic Regression), 0.728 (Random Forest) and 0.708 (KNN). 59.8% of applicants are classified correctly by all three models, 25.6% wrongly by all three, and only 14.6% split the models: the models make mostly the **same** mistakes. Errors concentrate where the credit score is low or middling (Logistic Regression error 35.7% and 39.6% in the two lowest quartiles vs 15.9% in the highest), but in those bands the model is no better than the best constant guess (31.8% and 40.3%; it is worse in the lowest band and less than one point better in the second). The model's skill comes from separating high from low credit scores; inside a band the other features add no usable signal. The 132 applicants that all three models get wrong were approved only 21% of the time, against 81% for those all get right, although their profiles are similar (credit score 602 vs 621; loan-to-income 0.87 vs 0.88). Their outcome contradicts their profile, which no better model could predict. KNN is weakest in the sparse low-credit bands (45.0% and 47.2% errors vs 35.7% and 39.6% for Logistic Regression) and for the small group of unemployed applicants (48.1% vs 37.0%).

4. Discussion: why the models perform differently

The test results in Part B rank the models, but a ranking does not explain itself. This section uses the experiments of Steps 10-12 to explain *why* each model performs as it does, how the features each model relies on differ, and how much better any model could realistically be.

4.1 Why Logistic Regression comes first

Logistic Regression has the highest cross-validated AUC (0.732) and the highest test AUC (0.756, with Random Forest at 0.755) of the three models. The experiments point to four reasons.

- **Its assumption fits the data.** Approval depends mainly on a few financial features that act additively (credit score, income, loan size). A model that is linear in the log-odds has the right shape for that. With only two inputs, credit score and loan-to-income, it reaches 0.746 AUC, the highest value in Figure 12, and adding the other twelve columns does not improve it.
- **It copes with useless columns.** Regularisation (C = 0.1) shrinks the coefficients of irrelevant columns almost to zero (Age +0.011, Education +0.006, Loan_Term_Months -0.030), so its AUC stays between 0.731 and 0.746 whatever the feature set. It is not perfect: Gender (-0.18) and the city dummies (about 0.10 to 0.16 in size) still receive small coefficients that are only noise.
- **It needs little data.** Its training and validation curves meet (gap 0.03 at full size), and the validation AUC is already 0.744 with 131 training rows against 0.737 with 329. With 412 training rows it has all the data it can use.
- **It is already close to the ceiling** of what this data allows (Section 4.5).

An honest caveat. The advantage is small. Logistic Regression beats Random Forest by +0.016 AUC and KNN by +0.037 on average, but the corrected p-values are 0.23 and 0.24, so the ranking is **not statistically established** with 515 applicants. The defensible conclusion is that Logistic Regression is at least as good as the other models here and is simpler and easier to explain, not that it is proven better.

4.2 Why KNN comes last

KNN has the lowest cross-validated AUC (0.654) and the lowest test accuracy (0.650). Four properties of the method explain it.

- **Distance treats every feature as equally important.** The distance is computed over all 14 features, but only credit score and a few financial features carry signal. The other columns add random distance and blur the neighbourhoods. Restricting KNN to the three financial features raises its AUC from 0.698 to 0.713 (+0.015), a repair similar to Random Forest (+0.019) and about twice Logistic Regression's (+0.008). Credit score alone is too little for it (0.661): it needs income and loan size as well.
- **It memorises.** With distance weighting its training AUC is 1.00 at every training size, because each training point is its own nearest neighbour. The gap to the validation AUC is 0.28, the largest of the three models.
- **It is weak where data are sparse.** In the two lowest credit-score quartiles it makes 45.0% and 47.2% errors, against 35.7% and 39.6% for Logistic Regression, and 48.1% against 37.0% for the small group of unemployed applicants. Local methods have few neighbours to learn from in such regions.
- **Its tuning may be incomplete.** The best setting, k = 31, is at the edge of the searched range (5, 11, 21, 31). A wider search with even more smoothing might raise KNN a little; the other models' best settings were inside their ranges.

4.3 Why Random Forest sits in the middle

Random Forest has a cross-validated AUC of 0.711 with default settings and 0.724 after tuning, and the best test recall of the three models (0.873).

- **It is flexible but data-hungry.** Its validation AUC rises from 0.676 with 65 rows to 0.728 with 329 and is still going up, while the gap to its training score narrows from 0.26 to 0.16. It is the one model that more data would probably help most; this was not tested with more data.

- **Tuning pushed it towards simplicity.** The default forest (depth 8) memorises the training data (training AUC 1.00 against a validation AUC of 0.711); the best tuned forest is shallow (depth 4). The validation curve in Figure 7 shows why: depth beyond 4 raises the training score but not the validation score.
- **Its main strength has little to work on.** Forests can capture interactions and non-linear effects. Here the data show no sign of them: a linear model matches the forest, and the forest is not better than Logistic Regression in any credit-score or loan-to-income band of the error analysis.
- **It also suffers from weak features.** With only the three financial features its AUC is 0.736, against 0.718 with all 14, and it spreads importance over correlated columns such as Age.

4.4 How feature importance varies across the models

Figure 11 and the tables in Step 10 show what each model relies on. The picture has one clear result and several differences that are mostly noise or measurement artefacts.

- **Agreement: credit score.** It is first for every model and every method. Shuffling it lowers the test AUC by 0.221, 0.179 and 0.199, whereas no other feature lowers any model's AUC by more than 0.023.
- **Different mechanisms use different secondary features.** Logistic Regression combines Annual_Income (0.023) and Loan_Amount (0.010) with opposite signs, which amounts to a loan-to-income ratio. A forest and KNN cannot form a ratio directly, so they pick up the same information through Age (0.011 and 0.013), which is correlated with income.
- **Correlated features share importance.** Age, income and loan amount move together (r = 0.74 between Age and income in Week 1). If a model can substitute one for another, shuffling one column hurts less, so the importance of each is understated. This is one reason why the rankings agree so little (Spearman 0.16 to 0.53 between models).
- **The importance measures themselves differ.** Coefficient size depends on the scale of the feature (the Unemployed dummy has a large coefficient but almost no effect on AUC); impurity importance favours columns with many distinct values (it ranks Loan_Term_Months and Education 5th and 6th although shuffling them changes nothing); and Logistic Regression's own coefficient and permutation rankings agree well (Spearman 0.79) while the forest's impurity and permutation rankings agree only moderately (0.41).
- **Noise limits what can be said.** The typical shuffle-to-shuffle noise is 0.005 to 0.013 AUC on a 103-row test set. Apart from credit score, almost every difference between models or features is inside about two times that noise. The only result that can be trusted is that credit score dominates, followed by the financial pair (income, loan size).

4.5 The ceiling: why no model can do much better here

Because the dataset was generated in Week 1, the rule that produced the approval label is known: a logistic function of credit score, income, the loan-to-income ratio and unemployment, from which each label was *drawn at random*. Applying that rule to the cleaned data gives a rough upper limit for any model. It is approximate, because the cleaning in Week 1 capped and filled some values, and it can only be computed for synthetic data; with real data the ceiling is unknown.

Measure	True rule (ceiling)	Logistic Regression
AUC on all 515 applicants	0.744	0.739 (out-of-fold)
Accuracy on all 515 applicants	0.697	0.699 (out-of-fold)
AUC on the 103 test applicants	0.759	0.756

- **The best model is at the ceiling** within the uncertainty of the estimate (about 0.02 AUC on 515 applicants). The modest scores in Part B reflect the data, not a weak model.
- **The mistakes are shared.** 59.8% of applicants are right for all three models and 25.6% are wrong for all three; only 14.6% split the models. Pairwise disagreement is only 7% to 12%. Different models fail on the same people.

- **The hard cases contradict their own profile.** The 132 applicants wrong for all three models were approved 21% of the time, against 81% for the group that is always right, although credit score (602 vs 621) and loan-to-income are almost the same. This is what random label noise looks like.
- **Inside a credit-score band the models add almost nothing.** In the lowest three quartiles the Logistic Regression error (35.7%, 39.6%, 27.6%) is within one percentage point of the best constant guess (40.3% and 28.4%) in the second and third quartile, and worse than it in the lowest (31.8%). The skill comes from separating high from low credit scores.

4.6 What would actually improve performance

- **More informative features, not more complex models.** Three model families with different mechanisms all stop at the same score and fail on the same applicants.
- **Domain features for the flexible models.** A loan-to-income ratio lifts every model above credit score alone, and removing uninformative columns helps KNN and Random Forest; both are cheap to do.
- **More data, if Random Forest is the goal.** Its learning curve is the only one still rising.
- **A decision threshold chosen by cost.** At threshold 0.5 the best model wrongly approves 47.5% of rejected applicants in the test set (Step 9 shows the trade-off).
- **Statistical care when comparing models.** Use repeated cross-validation and a paired test before claiming that one model is better; differences of 0.02 to 0.04 AUC are within noise here.

5. Summary of findings

Question	Answer
How many natural groups are there?	Weak evidence for 2: older, higher-income applicants versus younger, lower-income ones (silhouette 0.23). The data is one continuous cloud, not separate islands.
Do K-Means and Hierarchical clustering agree?	Moderately (ARI 0.34); they cut the cloud in different places.
Are the clusters meaningful?	Yes, in a limited way: approval is 68% vs 55%, although approval was not used for clustering.
Best classifier?	Logistic Regression: cross-validated AUC 0.738, test AUC 0.756, accuracy 0.718, F1 0.785.
Is it good?	Modestly. It beats the always-approve baseline clearly on AUC (+0.26) and accuracy (+0.11) but only slightly on F1 (+0.03). It is close to the best score the data allow.
Which models overfit?	KNN (training AUC 1.00 at every size) and the untuned Random Forest (1.00 vs 0.71 validation). Logistic Regression does not.
Why is Logistic Regression best?	The data are roughly linear in a few features, so a simple model fits; it ignores useless columns and needs little data. The lead is small (p = 0.23 to 0.24) and not statistically established.
Do the models use the same features?	They agree on credit score (AUC drop 0.18-0.22) and disagree on the rest: Logistic Regression uses income and loan size, KNN and Random Forest use Age as a proxy. Most secondary differences are within noise.
Could a better model do much better?	Probably not: the true rule gives AUC 0.744 against 0.739 for Logistic Regression, and 26% of applicants are wrong for all three models.
How to choose the threshold?	0.4 for fewer missed good applicants (recall 0.94), 0.6 for fewer wrongly approved ones (precision 0.78): a business decision.

6. Limitations and next steps

- **Weak cluster structure.** With a silhouette of about 0.23, the two clusters are a convenient split of one continuous group, not clearly separate populations. They should be used as descriptive segments, not as hard categories.

- **Small, synthetic data.** 515 rows and a 103-row test set mean that scores move by several percentage points with a different split; differences between the tuned models are within noise.
- **Scaling before splitting.** The Week 1 features were scaled on the whole dataset, which leaks a little information from the test rows. A Scikit-learn Pipeline would remove this.
- **Possible circularity.** Because the data is synthetic and the approval label was generated from income and credit score, the link between clusters and approval is partly built in. On real data it would need independent confirmation.
- **The ceiling is approximate.** It applies the known generating rule to cleaned data whose values were partly capped or filled in Week 1, and its AUC has an uncertainty of about 0.02. It exists only for synthetic data.
- **Importance and ablation results are indicative.** Permutation importance is computed on only 103 test rows, the ablation and learning-curve differences are 0.01 to 0.02 AUC, and the out-of-fold predictions use hyperparameters that were chosen on the training set. None of these differences was tested for significance, except the model comparison in Step 11c.
- **Next steps:** try DBSCAN and Gaussian mixtures, t-SNE or UMAP for visualisation, use a Pipeline, add calibration of probabilities, and repeat the evaluation on a real dataset.

Appendix: repository and how to run

```
week3-unsupervised-learning-evaluation/
  README.md
  requirements.txt
  data/loan_applicants_clean.csv          cleaned data from Week 1
  src/clustering_and_evaluation.py        PCA, clustering, evaluation, tuning, thresholds
  src/build_report.py                     builds this PDF
  results/                                tables (CSV), charts (PNG), clustered data, log
  models/best_classifier.joblib           saved best classifier
  report/                                 this PDF
```

To run: **pip install -r requirements.txt**, then **python3 src/clustering_and_evaluation.py** from the repository root.